



Tecnológico de Monterrey

Análisis y diseño de algoritmos avanzados

E1. Actividad Integradora 1

Integrantes

Abigail Pérez García | A01736815

Gadiro Cano Sánchez | A01737094

Fecha de entrega:

19 oct 2024

Desarrollo de la solución:

Para poder resolver este proyecto se realizó lo siguiente:

```
var nonAlphanumericRegex = regexp.MustCompile(`^[a-zA-Z0-9]+$`)

// Función para remover caracteres especiales
func onlyCharacters(str string) string {
    return nonAlphanumericRegex.ReplaceAllString(str, "")
}

// Función para remover las mayúsculas
func toLowerCase(s string) string {
    return strings.ToLower(s)
}

// Leer archivos y almacenar sus contenidos como un String "limpio"
func readFile(filename string) string {
    content, err := ioutil.ReadFile(filename)
    if err != nil {
        fmt.Println("Error reading the file:", err)
        os.Exit(1)
    }
    processedContent := toLowerCase(onlyCharacters(string(content)))
    return processedContent
}
```

Esta primera parte es la hermenéutica de nuestro problema, aquí se define la lógica para que solo acepte números y letras, excluyendo cualquier carácter especial, además de que convierte cualquier letra de Mayúscula a minúscula para evitar errores por esa parte. Además de aquí se encuentra la función que realiza la lectura del documento para poder leer todo su contenido.

```
// Construcción del Suffix Array
func buildSuffixArray(text string) []int {
    n := len(text)
    suffixArr := make([]int, n)
    for i := range suffixArr {
        suffixArr[i] = i
    }
    sort.Slice(suffixArr, func(i, j int) bool {
        return text[suffixArr[i]:] < text[suffixArr[j]:]
    })
    return suffixArr
}
```

Esta función crea un Suffix Array a partir de un texto dado.

Primero genera una lista de índices que representan todos los posibles sufijos del texto. Luego, ordena estos sufijos alfabéticamente utilizando los índices, y devuelve un objeto `SuffixArray` que contiene el texto original y el array de sufijos ordenados.

```
// Encontrar el prefijo común más largo entre 2 Strings
✓ func longestCommonPrefix(text string, i, j int) int {
    n := len(text)
    length := 0
    for i+length < n && j+length < n && text[i+length] == text[j+length] {
        length++
    }
    return length
}
```

Este código busca el prefijo común más largo entre dos posiciones (*i* y *j*) de un texto. Compara los caracteres de los sufijos a partir de esas posiciones, avanzando mientras los caracteres sean iguales, y devuelve la longitud de la coincidencia encontrada. El cual nos ayuda a comparar el texto y encontrar las coincidencias.

```
func longestCommonSubstring(text1, text2 string) int {
    combined := text1 + "#" + text2
    suffixArr := buildSuffixArray(combined)

    n1 := len(text1)
    longestLen := 0

    for i := 1; i < len(suffixArr); i++ {
        if (suffixArr[i-1] < n1 && suffixArr[i] > n1) || (suffixArr[i-1] > n1 && suffixArr[i] < n1) {
            lcp := longestCommonPrefix(combined, suffixArr[i-1], suffixArr[i])
            if lcp > longestLen {
                longestLen = lcp
            }
        }
    }
    return longestLen
}
```

Este código busca el substring más largo en común entre dos textos. Primero, combina los dos textos con un separador (`#`), construye un Suffix Array del texto combinado, y luego utiliza el prefijo común más largo (`lcp`) para comparar los sufijos de ambos textos. El algoritmo encuentra y devuelve la longitud del substring común más largo.

```

func calculateSimilarity(content1, content2 string, commonLength int) float64 {
    shorterLength := len(content1)
    if len(content2) < len(content1) {
        shorterLength = len(content2)
    }

    percentage := (float64(commonLength) / float64(shorterLength)) * 100
    return percentage
}

```

Esta función calcula el porcentaje de similitud entre dos textos, este recibe la longitud del substring común más largo (commonLength) y divide ese valor por la longitud del texto más corto, para luego multiplicar el resultado por 100.

Esto da el porcentaje de similitud entre los dos textos, en función del tamaño del substring en común.

```

func compareFiles(original string, others []string, comparisons *[][]string) {
    content1 := readFile(original)

    for _, other := range others {
        content2 := readFile(other)
        commonLength := longestCommonSubstring(content1, content2)

        if commonLength > 0 {
            percentage := calculateSimilarity(content1, content2, commonLength)
            // Guardar si la comparación es al menos 20% similar
            if percentage >= 20 {
                *comparisons = append(*comparisons, []string{
                    original,
                    other,
                    strconv.FormatFloat(percentage, 'f', 2, 64),
                })
            }
        }
    }
}

```

Esta función compara un archivo original con otros archivos, calculando la similitud entre ellos.

Si la similitud es mayor o igual al 20%, guarda la comparación (archivo original, archivo comparado y porcentaje de similitud) en una lista.

```

func main() {
    Originals := []string{
        "data/orig_taska.txt",
        "data/orig_taskb.txt",
        "data/orig_taskc.txt",
        "data/orig_taskd.txt",
        "data/orig_taske.txt",
    }

    TaskA := []string{
        "data/g0pA_taska.txt",
        "data/g0pB_taska.txt",
        "data/g0pC_taska.txt",
        "data/g0pD_taska.txt",
        "data/g0pE_taska.txt",
        "data/g1pA_taska.txt",
        "data/g1pB_taska.txt",
        "data/g1pD_taska.txt",
    }

    taskFilesMap := map[string][]string{
        "data/orig_taska.txt": TaskA,
        "data/orig_taskb.txt": TaskB,
        "data/orig_taskc.txt": TaskC,
        "data/orig_taskd.txt": TaskD,
        "data/orig_taske.txt": TaskE,
    }

    // Matriz de todas las comparaciones
    var comparisons [][]string

    // Comparar los "Originales" con los "Task" y guardar los datos
    for _, original := range Originals {
        compareFiles(original, taskFilesMap[original], &comparisons)
    }

    // Ordenar en base a porcentaje de similitud
    sort.Slice(comparisons, func(i, j int) bool {
        similarityI, _ := strconv.ParseFloat(comparisons[i][2], 64)
        similarityJ, _ := strconv.ParseFloat(comparisons[j][2], 64)
        return similarityI > similarityJ
    })

    fmt.Println("\nTop 10 Archivos más parecidos:")
    for i, comparison := range comparisons {
        if i >= 10 {
            break
        }
        fmt.Printf("Archivo base: %s, Archivo comparado: %s, Similitud: %s%%\n", comparison[0], comparison[1], comparison[2])
    }
}

```

Esta parte del código es la función que despliega todo, compara archivos originales con otros archivos de tareas utilizando un mapa que asocia cada archivo original con un conjunto de archivos correspondientes y realiza lo siguiente:

- Comparaciones: Para cada archivo original, se compara con los archivos de tareas asociados, calculando el porcentaje de similitud entre ellos.

- Almacenamiento de Resultados: Las comparaciones con al menos un 20% de similitud se guardan en una lista.
- Ordenamiento: La lista de comparaciones se ordena en función del porcentaje de similitud, de mayor a menor.
- Impresión del Top 10: Se imprimen las 10 comparaciones con mayor porcentaje de similitud.

Algoritmos empleados:

Suffix Array

- Complejidad temporal: $O(n^2 \log(n))$.
- Complejidad espacial: $O(n)$.

Por medio de la implementación de la función Suffix Array, tomamos el string que contiene todos los caracteres de un archivo y generamos el arreglo de sufijos correspondiente. Posteriormente, para posteriormente hacer uso del algoritmo LCS (Longest Common Substring) para comparar dichos sufijos y así determinar cuál de estos es más parecido al texto base. El objetivo es identificar las similitudes más relevantes, incluso si se han realizado modificaciones en el texto para ocultar el plagio.

Es importante destacar que la complejidad espacial del Suffix Array es $O(n)$, ya que solo requiere almacenar los sufijos del texto. Esto lo convierte en una herramienta eficiente para procesar y comparar grandes volúmenes de texto. A diferencia de un enfoque el cuál construyera todas las permutaciones posibles del texto (que tendría una complejidad mucho mayor), el Suffix Array permite comparar los textos de manera más rápida y con una menor demanda de recursos.

Longest Common Prefix

- Complejidad temporal: $O(n)$.
- Complejidad espacial: $O(1)$.

El algoritmo de Longest Common Prefix (LCP) se utiliza para calcular la longitud del prefijo común más largo entre dos sufijos consecutivos generados por el Suffix Array. Este algoritmo recorre los caracteres de ambos sufijos, comparándolos uno por uno. Si encuentra diferencias en algún punto, detiene el proceso, ya que no es necesario seguir comparando. El objetivo del LCP es complementar el algoritmo LCS, ya que permite comparar de forma eficiente los sufijos y determinar el grado de similitud entre los textos.

Longest Common Substring

- Complejidad temporal: $O((n_1+n_2)^2 \cdot \log(n_1+n_2))$
- Complejidad espacial: $O(n_1+n_2)$

Finalmente, el algoritmo de Longest Common Substring (LCS) utiliza el Suffix Array y el Longest Common Prefix (LCP) para encontrar el substring más largo en común entre los dos textos. Aquí, n_1 corresponde a la longitud del texto base, mientras que n_2 corresponde a la longitud del texto que está siendo comparado. La complejidad espacial es la suma de las longitudes de ambos textos, ya que tanto los sufijos como las comparaciones entre ellos dependen de estas longitudes.

El **LCS** busca los substrings comunes de mayor longitud entre los dos textos mediante la comparación de los sufijos y los prefijos comunes, recorriendo los textos de manera eficiente hasta haber procesado todas las posibles combinaciones. Este enfoque resulta particularmente útil para detectar similitudes o plagio entre textos, ya que puede identificar patrones que se repiten a pesar de modificaciones en los textos.